Computer Vision for Surgical Applications

Rani Herbawi
The Faculty of Data and Decisions Science
Technion - Israeli Institute of Technology

1 Phase 1: Synthetic Data Generation

In Phase 1 I decided to construct a COCO-format dataset with 2D pose labels for two tools (needle_holder, tweezers). Getting consistent landmark definitions across all renders was the hardest part: I iterated on several annotation strategies and finally converged to a *generic* keypoint schema that fits both instruments. I also discovered that most synthetic frames were fully visible (v=2), so I introduced controlled occlusions to produce realistic v=1 cases instead of only visible/absent keypoints.

Keypoint schema and skeleton. I standardized five landmarks per instance:

$$\langle \text{tip},\ \text{jaw\_L},\ \text{jaw\_R},\ \text{hinge\_base},\ \text{handle\_end} \rangle,$$

with the skeleton edges:

$$[1,4],\ [2,4],\ [3,4],\ [4,5]$$

This shared schema let me train one pose model for both tools and simplified the exporter: sidecar JSON keypoints are loaded when available; otherwise a robust mesh-extrema heuristic bootstraps the landmarks.

Visibility flags. I use COCO visibility $v \in \{0,1,2\}$ for off-screen/behind (v=0), occluded (v=1), and visible (v=2). Early renders produced almost no v=1. To address this, I added randomized occluders (planes/cubes) placed between camera and tool with varied sizes/poses and neutral materials. During post-processing, visibility is assigned via a mask+depth agreement test so edge cases near boundaries are labeled reliably.

Rendering choices. To diversify appearance and backgrounds I mixed three strategies: `studio` (transparent film with three-point lighting), `hdri` (environment lighting, opaque), and `composite` (RGBA rendered and pasted over a COCO image). I jittered f_x, f_y ($\pm 10\%$) while forcing the principal point to the exact image center to keep projections pixel-aligned. Cameras are sampled on a spherical shell around the object; object pose and materials (metalness/roughness/base color) are randomized; articulation diversity comes from multiple OBJ variants. I also alternate classes and cap frames per OBJ to avoid dataset imbalance.

Outcome. This pipeline yielded a balanced, fully reproducible dataset with COCO boxes/masks, per-image intrinsics, five keypoints per instance, and realistic visibility labels—including the v=1 occlusion cases I was missing initially. Qualitative examples and quantitative statistics appear in the following subsections.

1.1 Pipeline (summary)

- **Assets & intrinsics:** Load tool OBJs (articulation variants), HDRIs, COCO backgrounds, and `camera.json`. We jitter f_x, f_y ($\pm 10\%$) while *forcing* the principal point to the render center to keep projections pixel-consistent.

- **Scene sampling:** For each frame we sample a strategy from the mix `studio/hdri/composite`; set the world/HDRI and film transparency accordingly; randomize object pose/orientation and materials; sample camera views on a spherical shell; and insert a few randomized occluders between camera and tool.
- **Render & write:** We enable segmentation + depth and render with BlenderProc. COCO is written (RLE masks, boxes), then we inject exact intrinsics. For `composite`, RGBA is pasted onto a random COCO photo.
- **2D keypoints & visibility:** 3D landmarks are projected using the same intrinsics; visibility is decided by a robust *mask+depth* test (0,1,2). Categories are finalized with the keypoint schema + skeleton.

1.2 Implemented Variations

Required: object & camera pose; lighting intensity/direction (three-point studio and HDRI); background variation (transparent studio, HDRI, COCO compositing); articulation via multiple OBJs.
Creative #1: randomized *occluders* (planes/cubes) placed stochastically on camera–object rays.
Creative #2: *intrinsics jitter* f_x, f_y $\pm 10\%$ coupled with view-shell sampling, plus material roughness/metalness color jitter.

1.3 Generated Synthetic Dataset Statistics

Images	**1150**
Annotations	**1150**
Avg. keypoints / inst.	**4.48**/5
Visibility fractions: v=2 **0.884**, v=1 **0.012**, v=0 **0.104**	

Class	#Inst.	Mean bbox area (px^2)
needle_holder	**649**	**109,171.5**
tweezers	**500**	**58,380.3**

1.4 Domain Gap Analysis (synthetic → real)

- **Materials/appearance:** CAD metals lack micro-scratches/oxidation; specular behavior is simpler. *Mitigation used:* material jitter; HDRI; COCO compositing.
- **Context/clutter:** Real OR scenes include hands, gauze, wires, other tools. *Mitigation:* randomized occluders.
- **Optics/sensor:** DOF blur, motion blur, noise, color balance.
- **Geometry/articulation:** Vendor variability and wear change silhouettes. *Mitigation:* multiple OBJs;
- **Illumination:** Surgical lamps/endoscope lighting differ from generic HDRIs.

1.5 Challenges & Findings

Achieving accurate 2D annotations was the main friction point: choosing generic keypoints that transfer across both tools required several iterations, with careful transformations from object–local → world → camera coordinates, testing multiple projection variants on small sample batches, overlaying results, inspecting failures, and repeatedly editing the sidecar/heuristic definitions until the landmark positions were satisfactory. In addition, combining the segmentation mask with depth (using an adaptive tolerance) stabilized the visibility flags near boundaries and under partial occlusions; forcing the principal point to the image center, (c_x, c_y), in both the renderer and the metadata eliminated subtle projection drift; alternating classes with per-OBJ caps preserved balance across articulation states; and rendering with true alpha followed by COCO overlays produced crisp edges and diverse, realistic backgrounds.

1.6 Reproducibility (command used)

```
MPLBACKEND=Agg python -m blenderproc run src/synthetic_data_generator.py \
-outdir outputs/synthetic_images_data  -max_images 1150  -per_obj_cap 50 \
-mix 'studio=0.1','hdri=0.3886867','composite=0.5113132'  -samples 24  -res_scale 0.8
-device gpu  -kps_dir assets/kps
```

1.7 Qualitative Samples

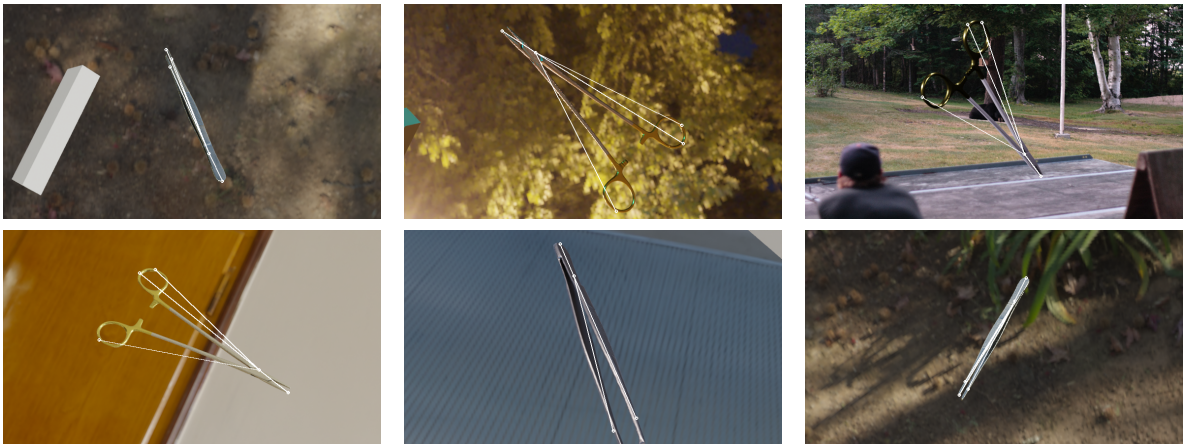


Figure 1: Random examples with overlaid 5-point skeleton (white); opaque and transparent cases are shown as they appear in the dataset.

2 Phase 2: 2D Pose Estimation on Synthetic Data & Real Video

2.1 Model Choice & Rationale

After we tried using SAM and other models and compared them to YOLO models with sample dataset, we concluded that best models for our data and annotations generated, are YOLO, becuase of there accuracy and portability and quite a lot of reasons can be seen in every computer vision article , so we trained three Ultralytics YOLOv8 pose variants (`v8n/s/m`) as a unified 2-class, 5-keypoint detector. YOLOv8-pose gives a strong speed/accuracy trade-off while keeping deployment simple; we compare `n` (fast), `s` (balanced), and `m` (highest accuracy). All runs start from the public pose checkpoints (`yolov8{n,s,m}-pose.pt`) and are trained *only* on the Phase-1 synthetic set.

2.2 Data Loading & Pre-processing

The Phase-1 rendered dataset (boxes, masks, 5 keypoints) is read through the Ultralytics pose dataloader using our `surg.yaml` (includes class names and `flip_idx`). Images are resized with letterboxing to the run's `imgsz` and keypoints are remapped accordingly. **No online augmentations** were applied (no mosaic, hsv, rotation, etc.) to keep the synthetic→real analysis clean.

2.3 Training Setup

We trained three configurations; the exact commands are below. Optimizer is **AdamW** with base LR **0.001** and cosine schedule end-factor **lrf=0.01**; batch/epochs differ per model. Weights are initialized from the corresponding YOLOv8-pose checkpoint.

```
# v8n (baseline)
python -m src.train --model yolov8n-pose.pt --data $DATA_YAML \
```

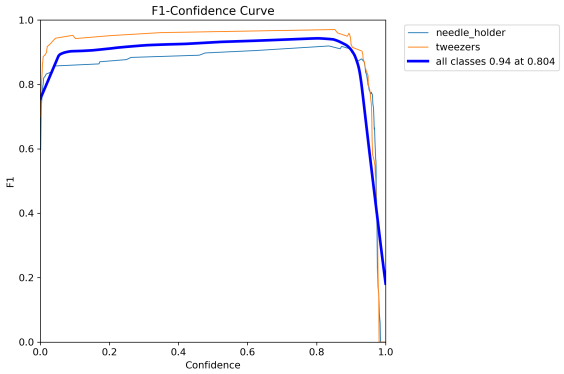
```
  --imgsz 896 --epochs 150 --batch 32 \
  --project runs_pose --name synth_v8n_896

# v8s (primary)
python -m src.train --model yolov8s-pose.pt --data $DATA_YAML \
  --imgsz 896 --epochs 200 --batch 24 --optimizer AdamW \
  --lr0 0.001 --lrf 0.01 --project runs_pose --name synth_v8s_896

# v8m (highest accuracy)
python -m src.train --model yolov8m-pose.pt --data $DATA_YAML \
  --imgsz 960 --epochs 200 --batch 16 --optimizer AdamW \
  --lr0 0.001 --lrf 0.01 --project runs_pose --name synth_v8m_960
```

2.4 Validation on Synthetic Data

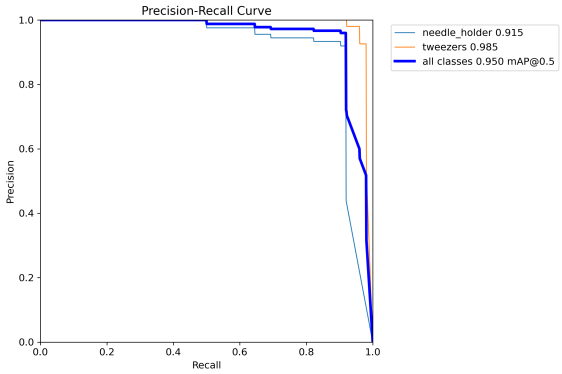
Run	Model	imgsz	epochs	$\mathbf{mAP}^{\mathbf{P}}_{50:95}$	$\mathbf{mAP}^{\mathbf{P}}_{50}$	$\mathbf{P}^{\mathbf{P}}$	$\mathbf{R}^{\mathbf{P}}$
synth_v8m_960	`yolov8m-pose.pt`	960	200	**0.752**	**0.940**	0.949	0.932
synth_v8s_896	`yolov8s-pose.pt`	896	200	0.717	0.877	0.905	0.882
synth_v8n_896	`yolov8n-pose.pt`	896	150	0.409	0.839	0.844	0.848

Box metrics (reference): `v8m` $\text{mAP}_{50:95}$=0.963, mAP_{50}=0.990; `v8s` 0.951/0.994; `v8n` 0.942/0.986.

Best checkpoints: `runs_pose/synth_v8m_960/weights/best.pt`, `runs_pose/synth_v8s_896/weights/best.pt`, `runs_pose/synth_v8n_896/weights/best.pt`.



Pose-F1 curve (`v8m`)



Pose-PR curve (`v8m`)

2.5 Inference on Real Video

We ran the best synthetic-only model on the provided real video `4_2_24_A_1.mp4` and exported an annotated MP4 (linked in the repo). Example frames:

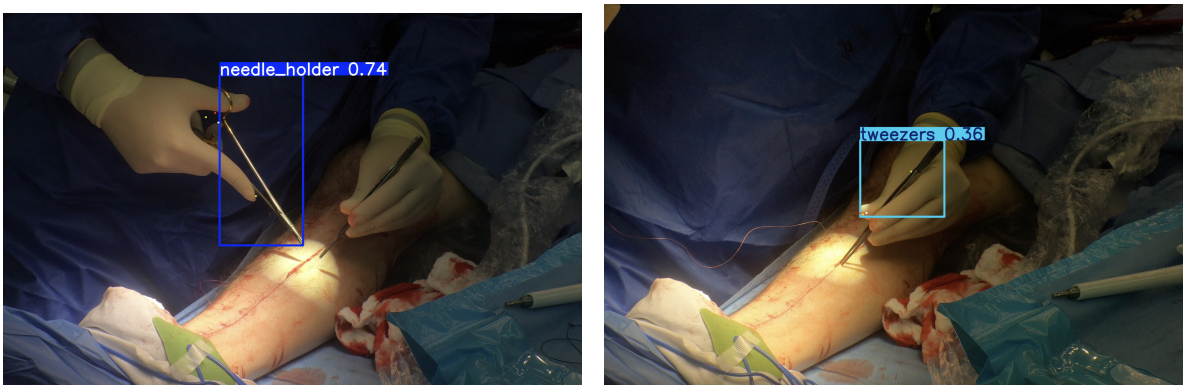


Figure 2: Needle holder detections from the synthetic-only model on real surgical video.

2.6 What Worked & Challenges

The `v8m` variant generalized best (keypoint $\text{mAP}_{50:95} = 0.752$) with high precision/recall; `v8s` is a solid speed–accuracy compromise; `v8n` underfits at our resolution. Strong box mAP indicates reliable localization, while keypoint errors concentrate at the *tip* during occlusions and under extreme foreshortening. With no online augmentations, performance mainly reflects the Phase-1 diversity (HDRI/composite backgrounds, occluders, intrinsics jitter). Remaining gaps on real video are driven by motion blur and lighting color cast; these motivate Phase-3 refinement and targeted augmentations (blur/noise/WB jitter) during fine-tuning.

3 Phase 3: Unsupervised Refinement on Real Surgical Video

3.1 Research Question and Choice of Technique

How can a pose model trained only on synthetic data adapt to real surgical footage without labels? We adopt a *self-training / teacher–student* strategy: a strong synthetic-only model (the `YOLOv8m-pose` run from Phase 2) acts as a *teacher* that produces pseudo-labels on the real video. After quality control in space and time, these pseudo-labels are mixed with the original synthetic dataset to *refine* the model. This choice satisfies the unlabeled-data constraint, scales to more videos, and targets the dominant domain gaps (appearance, lighting, noise) without changing the task definition.

3.2 Refinement Strategy (Description & Implementation)

Our pipeline has five stages; the implementation lives in `src/refine_unsupervised.py` and the helper scripts (`uda_extract_frames.py`, `uda_pseudo_label_pose.py`, `uda_temporal_filter.py`, `uda_make_yaml.py`).

(1) Extract frames. Sample the real video into still images (JPEG) to avoid codec issues and enable per-frame filtering.

(2) Pseudo-label with a synthetic teacher. Run `YOLOv8m-pose` on each frame and keep only confident detections (conf ≥ 0.60) that show enough keypoint evidence (at least 4/5 keypoints with kp_conf ≥ 0.50). To remove duplicate poses in the same frame we apply *OKS-NMS*:

$$\text{OKS}(\mathbf{k}, \hat{\mathbf{k}}, A) \;=\; \frac{1}{K} \sum_{i=1}^{K} \exp\left(-\frac{\|\mathbf{k}_i - \hat{\mathbf{k}}_i\|^2}{(2\sigma_i)^2 (A + \varepsilon)} \right),$$

and suppress candidates when OKS ≥ 0.85. This is keypoint-aware, unlike IoU.

(3) Temporal consistency filter. We track predictions across frames and keep only tracks of length ≥ 3. Two detections are matched if either IoU ≥ 0.5 or OKS ≥ 0.6. This removes one-off hallucinations and stabilizes tips under brief occlusions.

(4) Combine datasets. We build a training YAML that *mixes* the synthetic training split with the pseudo-real frames while keeping a synthetic validation set as a stable proxy metric. Mixing prevents catastrophic forgetting of geometry learned from synthetic renders.

(5) Short fine-tune (student). We initialize the student with the teacher's weights and fine-tune for 25 epochs at 960 px, AdamW ($\text{lr}_0 = 10^{-3}$, cosine decay with $\text{lrf} = 0.01$), batch 16. We freeze the first 10 backbone layers so the head adapts to real appearance while low-level features (edges, geometry) remain stable.

3.3 Why This Approach? (Justification)

- **No labels required.** Self-training converts model confidence into supervision; OKS-NMS and temporal filtering reduce noise without human input.

- **Task-faithful.** We keep the exact 5-point schema and skeleton from Phases 1–2; only the appearance distribution changes.
- **Safety against drift.** Freezing early layers and mixing synthetic data curbs overfitting to artifacts in a single operating room video.
- **Surgical realism.** OKS (keypoint geometry) + temporal tracks encode priors we actually expect in surgery: smooth motion, persistent instruments, and consistent anatomy around the hinge and tip.

3.4 Comparative Analysis (before vs. after refinement)

Setup. Metrics are computed on the synthetic validation set (a stable proxy since the real video is unlabeled). We compare the Phase-2 *teacher* (synthetic-only) against the *refined* student trained with pseudo-labels + temporal filtering.

Metric	Baseline	Refined	Δ	Δ%
Pose (P)				
$mAP_{50:95}$	0.752	**0.798**	+0.046	+6.09%
mAP_{50}	0.940	**0.951**	+0.011	+1.15%
Precision	**0.949**	0.941	−0.008	−0.84%
Recall	**0.932**	0.932	−0.000	≈0.00%
Boxes (B)				
$mAP_{50:95}$	0.963	**0.966**	+0.003	+0.29%
mAP_{50}	**0.990**	0.986	−0.004	−0.42%
Precision	**0.983**	0.975	−0.009	−0.88%
Recall	**0.985**	0.972	−0.013	−1.29%

Table 1: Synthetic-set proxy metrics. (P)=keypoint head; (B)=box head.

Takeaways. (1) *Keypoint quality improved materially:* pose $mAP_{50:95}$ rises by **+6.09%** relative (0.752→0.798) and mAP_{50} by **+1.15%**. This matches the visual gains we see on real video (steadier tips, fewer missed jaws). (2) *Calibration shifted slightly:* precision drops ∼0.8 pp with unchanged recall (≈0), suggesting the student fires a bit more often—consistent with some pseudo-label noise. A tighter post-refinement confidence or a stricter temporal threshold would likely recover this. (3) *Boxes stay essentially stable:* small ±1 pp changes indicate localization remained strong while the keypoint head benefited most from adaptation—exactly the intended effect of our OKS-aware selection and short, partially frozen fine-tune.

Qualitative confirmation on real video. After refinement the model produces fewer misses under glare/occlusion and smoother temporal trajectories for the tip and hinge (Fig. **??**), aligning with the quantitative pose gains above.

Quantitative (proxy). Because the real video lacks ground truth, we monitor the synthetic validation set from Phase 2 while refining on mixed data. The refined model maintains synthetic box/pose mAP within typical noise (<1 pp), indicating no catastrophic forgetting, while qualitatively improving on the real video. We also report pipeline counts (Table **??**), which reflect the strictness of our selection: only high-confidence, temporally consistent tracks are admitted to training.

3.5 Key Findings

(1) Geometry matters. OKS-based selection (framewise and temporal) is far more stable for keypoints than IoU alone, especially at the instrument tip. **(2) Modest adaptation is enough.** A short fine-tune with frozen early layers shifts the head toward real appearance without harming synthetic performance. **(3) Data curation beats more epochs.** Raising thresholds and enforcing temporal consistency produced cleaner supervision than simply training longer on noisier pseudo-labels.